

Lesson 02 Demo 01

Building a CrewAI-Powered Agent with OpenAI

Objective: To demonstrate how to build a multi-agent system using CrewAI that intelligently identifies queries requiring real-time information and retrieves the most up-to-date data accordingly

Scenario: A product manager at a digital knowledge platform needs to manage an increasing volume of user queries, many of which require real-time updates and dynamic information. At present, these queries are reviewed and answered manually, leading to delays, inconsistencies, and rising operational costs. The goal is to automate the query-handling process using a multi-agent system that can identify when real-time data retrieval is needed and provide users with fast, accurate, and context-aware responses while maintaining efficiency and cost control.

Tools required: Visual Studio Code

Prerequisites: Azure OpenAI key

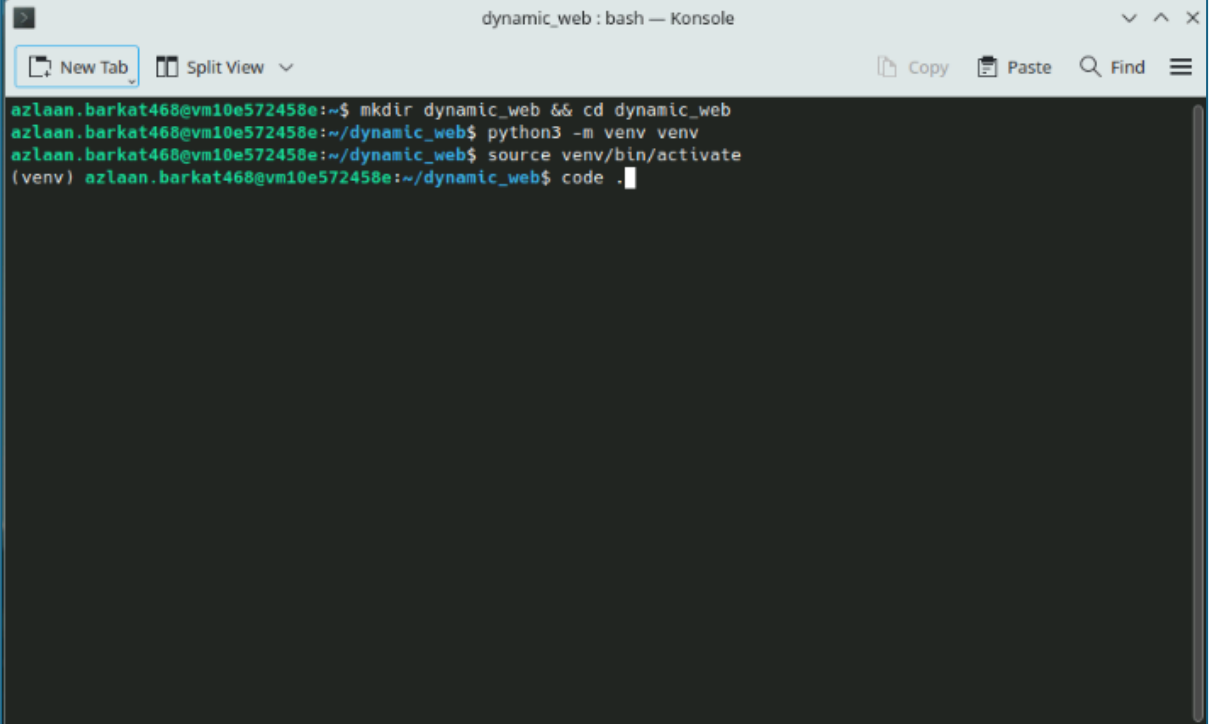
Steps to be followed:

1. Set up your local environment in Visual Studio Code
2. Configure the Tavily API Key and Azure OpenAI model
3. Define agents and tasks
4. Assemble and run the crew

Step 1: Set up your local environment in Visual Studio Code

1.1 Open the terminal in your system and run the following commands:

```
mkdir dynamic_web && cd dynamic_web  
python3 -m venv venv  
source venv/bin/activate  
code .
```



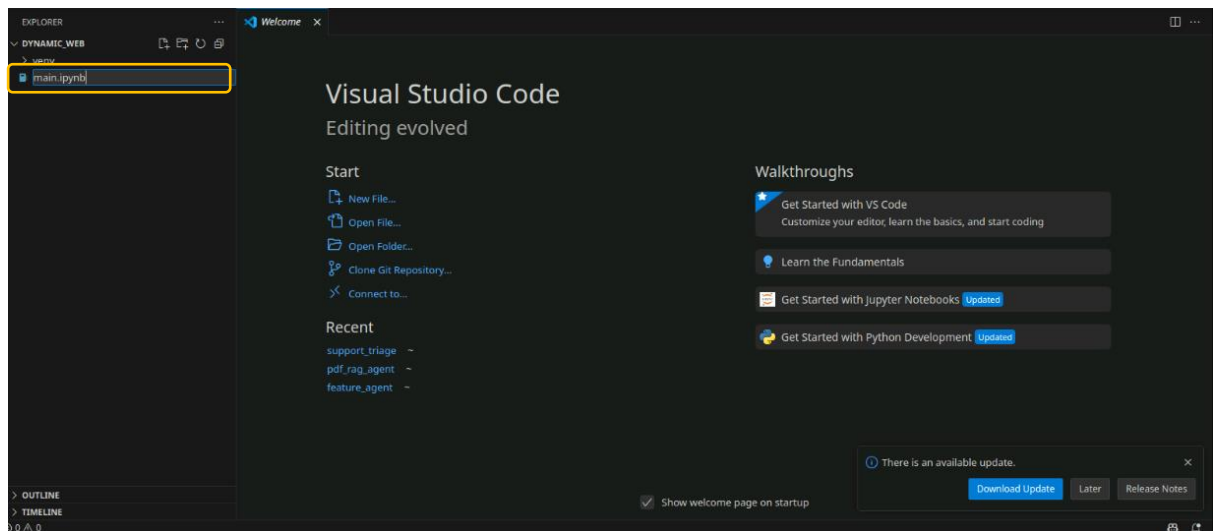
The screenshot shows a terminal window titled "dynamic_web : bash — Konsole". The terminal displays the following commands and their output:

```
azlaan.barkat468@vm10e572458e:~$ mkdir dynamic_web && cd dynamic_web  
azlaan.barkat468@vm10e572458e:~/dynamic_web$ python3 -m venv venv  
azlaan.barkat468@vm10e572458e:~/dynamic_web$ source venv/bin/activate  
(venv) azlaan.barkat468@vm10e572458e:~/dynamic_web$ code .
```

The terminal window has a standard interface with a title bar, a menu bar (File, Edit, View, Window, Help), and a toolbar (New Tab, Split View, Copy, Paste, Find, and a hamburger menu). The background is dark, and the text is light green.

Note: This creates a virtual environment inside the dynamic_web folder and opens the project in VS Code.

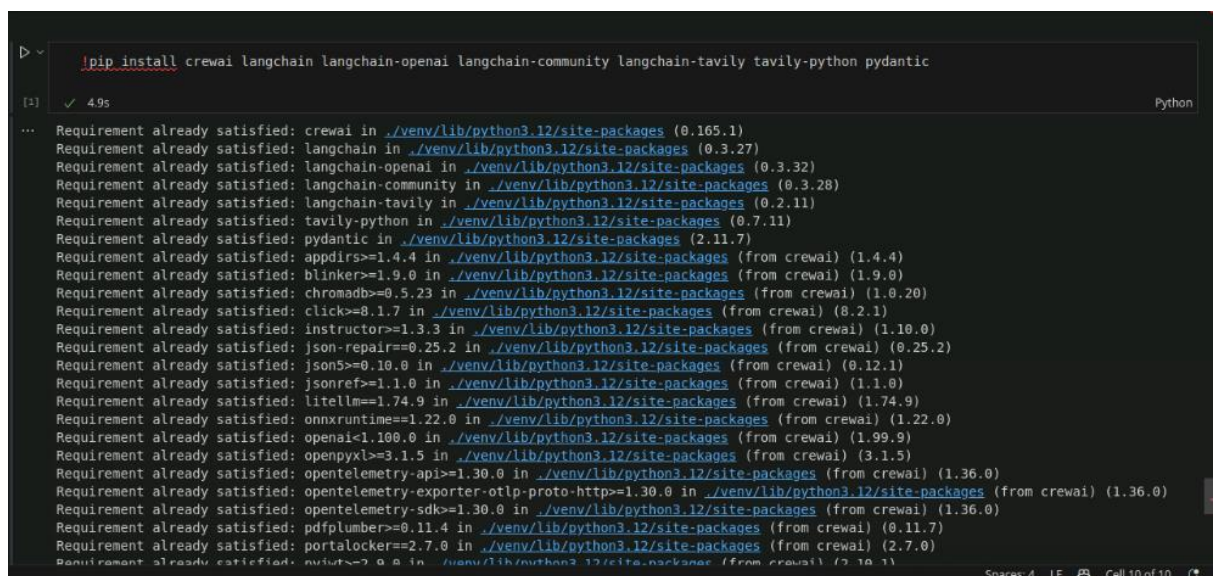
1.2 Create a new file and name it **main.ipynb**



1.3 In the first 2 cells, write and run the following commands to install the required packages:

!pip install crewai langchain langchain-openai langchain-community langchain-tavily tavily-python pydantic

!pip install "crewai[azure-ai-inference]"



```
main.ipynb > #
Generate + Code + Markdown ▶ Run All ↺ Restart 🗑 Clear All Outputs | 📄 Jupyter Variables ☰ Outline ... venv (Python 3.12.3)

!pip install "crewai[azure-ai-inference]"

(e) ✓ 54s Python

... Requirement already satisfied: crewai[azure-ai-inference] in ./venv/lib/python3.12/site-packages (1.10.1)
Requirement already satisfied: aiosqlite==0.21.0 in ./venv/lib/python3.12/site-packages (from crewai[azure-ai-inference]) (0.21.0)
Requirement already satisfied: appdirs==1.4.4 in ./venv/lib/python3.12/site-packages (from crewai[azure-ai-inference]) (1.4.4)
Requirement already satisfied: chromadb==1.1.0 in ./venv/lib/python3.12/site-packages (from crewai[azure-ai-inference]) (1.1.1)
Requirement already satisfied: click==8.1.7 in ./venv/lib/python3.12/site-packages (from crewai[azure-ai-inference]) (8.1.8)
Requirement already satisfied: httpx==0.28.1 in ./venv/lib/python3.12/site-packages (from crewai[azure-ai-inference]) (0.28.1)
Requirement already satisfied: instructor==1.3.3 in ./venv/lib/python3.12/site-packages (from crewai[azure-ai-inference]) (1.14.5)
Requirement already satisfied: json-repair==0.25.2 in ./venv/lib/python3.12/site-packages (from crewai[azure-ai-inference]) (0.25.3)
Requirement already satisfied: json5==0.10.0 in ./venv/lib/python3.12/site-packages (from crewai[azure-ai-inference]) (0.10.0)
Requirement already satisfied: jsonref==1.1.0 in ./venv/lib/python3.12/site-packages (from crewai[azure-ai-inference]) (1.1.0)
Requirement already satisfied: lancedb==0.29.2 in ./venv/lib/python3.12/site-packages (from crewai[azure-ai-inference]) (0.29.2)
Requirement already satisfied: mcp==1.26.0 in ./venv/lib/python3.12/site-packages (from crewai[azure-ai-inference]) (1.26.0)
Requirement already satisfied: openai==1.83.0 in ./venv/lib/python3.12/site-packages (from crewai[azure-ai-inference]) (2.26.0)
Requirement already satisfied: openpyxl==3.1.5 in ./venv/lib/python3.12/site-packages (from crewai[azure-ai-inference]) (3.1.5)
Requirement already satisfied: opentelemetry-api==1.34.0 in ./venv/lib/python3.12/site-packages (from crewai[azure-ai-inference]) (1.34.1)
Requirement already satisfied: opentelemetry-exporter-otlp-proto-http==1.34.0 in ./venv/lib/python3.12/site-packages (from crewai[azure-ai-inference]) (1.34.1)
Requirement already satisfied: opentelemetry-sdk==1.34.0 in ./venv/lib/python3.12/site-packages (from crewai[azure-ai-inference]) (1.34.1)
Requirement already satisfied: pdfplumber==0.11.4 in ./venv/lib/python3.12/site-packages (from crewai[azure-ai-inference]) (0.11.9)
Requirement already satisfied: portalocker==2.7.0 in ./venv/lib/python3.12/site-packages (from crewai[azure-ai-inference]) (2.7.0)
Requirement already satisfied: pydantic-settings==2.10.1 in ./venv/lib/python3.12/site-packages (from crewai[azure-ai-inference]) (2.10.1)
Requirement already satisfied: pydantic==2.11.9 in ./venv/lib/python3.12/site-packages (from crewai[azure-ai-inference]) (2.11.10)
Requirement already satisfied: pyjwt==3.2.0 in ./venv/lib/python3.12/site-packages (from crewai[azure-ai-inference]) (2.11.0)
Requirement already satisfied: python-dotenv==1.1.1 in ./venv/lib/python3.12/site-packages (from crewai[azure-ai-inference]) (1.1.1)
Requirement already satisfied: regex==2026.1.15 in ./venv/lib/python3.12/site-packages (from crewai[azure-ai-inference]) (2026.1.15)
Requirement already satisfied: textual==7.5.0 in ./venv/lib/python3.12/site-packages (from crewai[azure-ai-inference]) (8.1.1)
...
Using cached azure-core-1.38.2-py3-none-any.whl (217 kB)
```

```
Generate + Code + Markdown | Run All | Restart | Clear All Outputs | Jupyter Variables | myenv (Python 3.12.3)
Requirement already satisfied: propcache>=0.2.0 in ./myenv/lib/python3.12/site-packages (from aiohttp>=3.10->litellm) (0.2.0)
Requirement already satisfied: yarl<2.0,>=1.17.0 in ./myenv/lib/python3.12/site-packages (from aiohttp>=3.10->litellm) (1.17.0)
Requirement already satisfied: anyio in ./myenv/lib/python3.12/site-packages (from httpx>=0.23.0->litellm) (4.1.0)
Requirement already satisfied: certifi in ./myenv/lib/python3.12/site-packages (from httpx>=0.23.0->litellm) (2024.7.4)
Requirement already satisfied: httpcore==1.* in ./myenv/lib/python3.12/site-packages (from httpx>=0.23.0->litellm) (1.0.6)
Requirement already satisfied: idna in ./myenv/lib/python3.12/site-packages (from httpx>=0.23.0->litellm) (3.10)
Requirement already satisfied: h11>=0.16 in ./myenv/lib/python3.12/site-packages (from httpcore==1.*->httpx>=0.23.0->litellm) (0.14.0)
...
Requirement already satisfied: packaging>=20.9 in ./myenv/lib/python3.12/site-packages (from huggingface-hub<1.0,>=0.23.0->litellm) (24.1)
Requirement already satisfied: pyyaml>=5.1 in ./myenv/lib/python3.12/site-packages (from huggingface-hub<1.0,>=0.23.0->litellm) (6.0.2)
Requirement already satisfied: charset_normalizer<4,>=2 in ./myenv/lib/python3.12/site-packages (from requests>=2.26.0->litellm) (3.3.2)
Requirement already satisfied: urllib3<3,>=1.21.1 in ./myenv/lib/python3.12/site-packages (from requests>=2.26.0->litellm) (2.2.3)
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...

import os
import requests
from crewai.llm import LLM
from crewai import Agent, Task, Crew
from crewai.tools import BaseTool

[2] ✓ 12.3s Python
```

Step 2: Configure the Tavily API key and Azure OpenAI model

2.1 Use the Tavily API key to build a web search using the following code:

```
# Tavily API Key
TAVILY_API_KEY = "tvly-dev-1uT9rc-
X1i2A54uzUo18Hhz6BL1UjY8KMsU5aSCBbu3VRbNkr"

# ---- Custom CrewAI Tool for Web Search ----
class TavilySearchTool(BaseTool):
    name: str = "Web Search"
    description: str = "Search the web for recent information."

    def _run(self, query: str):
        url = "https://api.tavily.com/search"

        payload = {
            "api_key": TAVILY_API_KEY,
            "query": query,
            "max_results": 3
        }

        response = requests.post(url, json=payload)
        data = response.json()

        results = []
        for r in data["results"]:
```

```

        results.append(f"{r['title']} - {r['url']}")

    return "\n".join(results)

```

```
search_tool = TavilySearchTool()
```

```

# Tavily API Key
TAVILY_API_KEY = "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"

# ---- Custom CrewAI Tool for Web Search ----
class TavilySearchTool(BaseTool):
    name: str = "Web Search"
    description: str = "Search the web for recent information."

    def _run(self, query: str):
        url = "https://api.tavily.com/search"

        payload = {
            "api_key": TAVILY_API_KEY,
            "query": query,
            "max_results": 3
        }

        response = requests.post(url, json=payload)
        data = response.json()

        results = []
        for r in data["results"]:
            results.append(f"{r['title']} - {r['url']}")

        return "\n".join(results)

search_tool = TavilySearchTool()

```

Note: Please create your own Tavily key and use it in the above code block.

2.2 Use the Azure OpenAI API key to build an LLM using the following code:

```

# ---- Azure LLM ----
llm = LLM(
    model="azure/gpt-5-mini",

    api_key="2ABecnfxzhRg4M5D6pBKiqxXVhmGB2WvQ0aYKkbTCPsj0JLKsZPfJQQJ99
BDAC77bzfXJ3w3AAABACOGi3sC",
    base_url="https://openai-api-management-gw.azure-api.net/",
    api_version="2024-12-01-preview",
    is_litellm=True,
    temperature=0,
    max_completion_tokens=3500
)

```



```
# ---- Azure LLM ----
llm = LLM(
    model="azure/gpt-5-mini",
    api_key="2ABecnfxzhRg4M5D6pBKiqxXVhmGB2WvQ0aYKkbTCPsj0JLKsZPfJQQJ99BDAC77bzfXJ3w3AAABACOGi3sC",
    base_url="https://openai-api-management-gw.azure-api.net/",
    api_version="2024-12-01-preview",
    is_litellm=True,
    temperature=0,
    max_completion_tokens=3000
)
```

Step 3: Define agents and tasks

3.1 Use the following code to create agents:

```
# ---- Agents ----
researcher = Agent(
    role="AI Researcher",
    goal="Find the latest advancements in AI for FMCG using web search",
    backstory="Expert in AI research trends who gathers real-time information from the web.",
    verbose=True,
    allow_delegation=False,
    llm=llm,
    max_iter=1,
    tools=[search_tool]
)

writer = Agent(
    role="Technical Writer",
    goal="Summarize research into an executive report",
    backstory="Expert in executive summaries.",
    verbose=True,
    allow_delegation=False,
    llm=llm,
    tools=[search_tool]
)
```

```

# Researcher agent
researcher = Agent(
    role="AI Researcher",
    goal="Find the latest advancements in AI for FMCG",
    backstory="You are an expert in artificial intelligence and stay updated with the latest research trends in FMCG.",
    verbose=True,
    allow_delegation=False,
    llm=llm,
    max_iter=1,
    tools=[search_tool]
)

# Writer agent
writer = Agent(
    role="Technical Writer",
    goal="Summarize research into an executive report",
    backstory="You are an experienced technical writer with expertise in summarizing research for executives.",
    verbose=True,
    allow_delegation=False,
    llm=llm,
    tools=[search_tool]
)

```

3.2 Define the research and writing tasks:

---- Tasks ----

```

task_research = Task(
    description="Search the web and identify the top 3 recent advancements in AI for FMCG.",
    expected_output="Detailed notes explaining three recent AI advancements in FMCG with examples.",
    agent=researcher
)

```

```

task_write = Task(
    description="""
Write a concise executive summary using the research notes.

```

Requirements:

- Maximum 100 words
- Use bullet points
- Focus only on the 3 key advancements

```

""",
    expected_output="Executive summary of AI advancements in FMCG.",
    agent=writer,
    context=[task_research]
)

```



```
# ---- Tasks ----
task_research = Task(
    description="Search the web and identify the top 3 recent advancements in AI for FMCG.",
    expected_output="Detailed notes explaining three recent AI advancements in FMCG with examples.",
    agent=researcher
)

task_write = Task(
    description="""
Write a concise executive summary using the research notes.

Requirements:
- Maximum 100 words
- Use bullet points
- Focus only on the 3 key advancements
""",
    expected_output="Executive summary of AI advancements in FMCG.",
    agent=writer,
    context=[task_research]
)
```

Step 4: Assemble and run the crew

4.1 Use the code given below to arrange all the components of the crew and run it:

```
# ---- Crew ----
crew = Crew(
    agents=[researcher, writer],
    tasks=[task_research, task_write],
    verbose=True
)
```

```
result = await crew.kickoff_async()
```

```
print("\nFinal Output:\n")
print(result.raw)
```

```
# ---- Crew ----
crew = Crew(
    agents=[researcher, writer],
    tasks=[task_research, task_write],
    verbose=True
)

result = await crew.kickoff_async()

print("\nFinal Output:\n")
print(result.raw)
```

The program starts generating a summary report, as shown in the following screenshots:

```
... Crew Execution Started ...

Crew Execution Started
Name:
crew
ID:
bf947ad6-f66f-47c0-9b00-d9ac1c94cb5d

...

... Task Started ...

Task Started
Name: Search the web and identify the top 3 recent advancements in AI for FMCG.
ID: d982c1b2-72b5-42be-8f18-550b10bdad05

...

... Agent Final Answer ...

Agent: AI Researcher

Final Answer:
Top 3 recent AI advancements in FMCG (detailed notes with examples)

1) AI-powered probabilistic & causal demand forecasting (next-gen forecasting for planning and supply)
- What's changed recently
  - Shift from point-forecast ML to probabilistic forecasting (models that output full demand distributions and prediction intervals rather than single-point forecasts). This enables better risk-aware decisions (inventory buffers, service-level tradeoffs) and scenario planning.
  - Increased use of causal and hybrid models that combine time-series ML with causal drivers (promotions, macro/market signals, weather, competitor behavior) to isolate effects and support "what-if" simulations.
  - Faster model cycles and MLOps/augmentation so forecasting models are refreshed more frequently and tied directly into S&OP/IBP systems (shorter lead times from data to actionable plan).
  - Adoption of external alternative data (mobility, search trends, social, syndicated POS) and transfer learning across similar SKUs/regions to improve cold-start forecasts for new SKUs/markets.
- Why it matters for FMCG
  - FMCG margins and service levels are highly sensitive to stockouts and excess inventory; probabilistic forecasts allow optimization that trades off waste, storage costs and lost sales in a principled way.
  - Enables more robust promotions planning and improved allocation in omni-channel environments.
- Vendor & real-world examples
  - Blue Yonder / o9 Solutions / other supply-chain AI vendors: recent product updates and case studies (2024-25) emphasize probabilistic forecasting, scenario planning, and embedding ML models into planning workflows. (See vendor release notes and o9 case studies describing demand planning deployments for food & beverage customers.)
  - Large FMCG players (e.g., Nestlé, Coca-Cola, major retailers) are documented moving to ML-based demand
```

```
...
Final Output:
- Probabilistic & causal demand forecasting – next-gen planning: recent shift from point forecasts to probabili
- Generative AI for creative production, personalization & campaign automation: mature text/image/video models
- Computer vision & shelf intelligence for real-time in-store execution: edge CV and improved sensing deliver r
...

Crew Completion

Crew Execution Completed
Name:
crew
ID:
bf947ad6-f66f-47c0-9b00-d9ac1c94cb5d
Final Output: - Probabilistic & causal demand forecasting – next-gen planning: recent shift from point
forecasts to probabilistic and causal/hybrid models (with alternative data and faster MLOps) enables
scenario-based, risk-aware inventory and promotions planning; delivers lower forecast error, improved
safety-stock sizing and faster S&OP turnaround; in production with vendors like Blue Yonder/o9 and major
FMCG–requires strong data integration, governance and planner adoption.
```

By following these steps, you have successfully built a CrewAI-powered multi-agent system that intelligently routes and processes user queries, determines when real-time web data is needed, and retrieves the latest results via the Tavily Search Tool.